

SISTEMAS DE COMPUTADORES – 2023/2024
Exame Época Especial

Versão A

Apenas autorizada a consulta da folha oficial
Duração: 2h

Nome: _____ Nº _____

Folha de Respostas
(14/20 valores)

NOTAS:

1. Em todas as questões deverá assinalar apenas uma resposta.
2. Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.
3. Apenas as respostas às questões de escolha múltipla assinaladas na Folha de Respostas serão consideradas.
4. A parte prática deve ser respondida numa folha separada devidamente identificada.
5. Devem ser entregues todas as folhas do exame.

1 - a) ☐ b) ☐ c) ☐ d) ☐

2 - a) ☐ b) ☐ c) ☐ d) ☐

3 - a) ☐ b) ☐ c) ☐ d) ☐

4 - a) ☐ b) ☐ c) ☐ d) ☐

5 - a) ☐ b) ☐ c) ☐ d) ☐

6 - a) ☐ b) ☐ c) ☐ d) ☐

7 - a) ☐ b) ☐ c) ☐ d) ☐

8 - a) ☐ b) ☐ c) ☐ d) ☐

9 - a) ☐ b) ☐ c) ☐ d) ☐

10 - a) ☐ b) ☐ c) ☐ d) ☐

11 - a) ☐ b) ☐ c) ☐ d) ☐

12 - a) ☐ b) ☐ c) ☐ d) ☐

13 - a) ☐ b) ☐ c) ☐ d) ☐

14 - a) ☐ b) ☐ c) ☐ d) ☐

15 - a) ☐ b) ☐ c) ☐ d) ☐

16 - a) ☐ b) ☐ c) ☐ d) ☐

17 - a) ☐ b) ☐ c) ☐ d) ☐

18 - a) ☐ b) ☐ c) ☐ d) ☐

19 - a) ☐ b) ☐ c) ☐ d) ☐

20 - a) ☐ b) ☐ c) ☐ d) ☐

1 – Em Linux, quando um processo cria outro invocando a chamada ao sistema (*system call*) *fork()*, qual das seguintes características do processo pai não é copiada para o processo filho:

- a) Código executável.
- b) *Stack*.
- c) *Heap*.
- d) Valor retornado pelo *fork()*.

2 – Se um processo filho modificar o valor de uma variável num programa em C, qual das seguintes afirmações descreve o seu efeito?

- a) A modificação afetará apenas o processo filho.
- b) A modificação afetará tanto o processo filho como o processo pai.
- c) A modificação apenas afetará o processo pai se a variável for global.
- d) A modificação não terá nenhum efeito porque as variáveis não podem ser modificadas após a criação de um processo filho.

3 – Com a execução concorrente de processos num sistema com um único CPU podemos ter:

- a) Um processo em execução, enquanto outro está bloqueado numa operação de sincronização usando um mecanismo de espera ativa.
- b) Um processo em execução, enquanto outros estão bloqueados numa operação de sincronização usando um mecanismo de espera passiva.
- c) Mais do que um processo em execução simultânea, quer existam ou não processos bloqueados em operações de sincronização usando mecanismos de espera passiva.
- d) Apenas um processo bloqueado em operações de sincronização usando mecanismos de espera passiva, quer existam ou não processos em execução.

4 – No diagrama de transições de estado de um processo discutido nas aulas, a transição de “pronto a executar” para “em execução” indica que:

- a) Um processo em execução foi preemptado por outro processo escolhido pelo escalonador.
- b) Um processo deixou de estar bloqueado num semáforo.
- c) A espera passiva do processo pela operação de I/O ou evento terminou.
- d) Um novo processo acabou de ser criado.

5 – Num sistema operativo com paginação, um endereço lógico usado numa instrução:

- a) Corresponde à posição exata da instrução na memória física, determinada no momento da carga do programa.
- b) É um índice para uma tabela de páginas, que contém os endereços físicos das páginas correspondentes.
- c) É um deslocamento (*offset*) dentro de uma página, combinado com o número da página, que é utilizado pela MMU para calcular o endereço físico.
- d) É um endereço virtual que é traduzido para um endereço físico pela MMU, mas não possui relação direta com a organização da memória física.

6 – Num sistema operativo, o Process Control Block (PCB):

- a) Contém informação que permite a troca de contexto entre processos.
- b) É utilizado para facilitar a comunicação entre processos.
- c) É utilizado para armazenar o código fonte de um processo.
- d) Serve como interface entre o sistema operativo e o processo para a execução de chamadas ao sistema.

7 – A comunicação entre processos permite:

- a) A troca de dados e a sincronização de ações, apenas quando os processos partilham o mesmo espaço de endereçamento.
- b) A troca de dados e a sincronização das ações, mesmo sem partilha do mesmo espaço de endereçamento.
- c) Apenas a troca de dados, sem sincronização das suas ações, independentemente da partilha do espaço de endereçamento.
- d) Apenas a sincronização das ações, sem troca de dados, independentemente da partilha do espaço de endereçamento.

- 8** – Em programação concorrente, uma secção crítica é uma:
- a) Parte do programa em que são acedidos dados potencialmente partilhados por vários processos que devem ser alterados em exclusão mútua.
 - b) Parte do programa em que o processo requisita ao sistema operativo a reserva de mais memória de forma dinâmica.
 - c) Parte do programa em que um *bug* causa garantidamente o término do processo.
 - d) Parte do programa que é executada em *kernel space*.
- 9** – Uma solução eficiente para o problema da secção crítica deve garantir:
- a) Diferentes prioridades para os processos, envelhecimento (*aging*), ausência de interbloqueio (*deadlock*).
 - b) Acesso exclusivo, preempção, progressão.
 - c) Acesso exclusivo, progressão, espera limitada.
 - d) Ausência de preempção, privação de recursos (*resource starvation*), inversão de prioridades.
- 10** – Quando um processo é bloqueado ao chamar a função `sem_wait(&S)` num semáforo *S*, assumindo que *S* tinha o valor 0 antes da chamada, o processo fica no estado de:
- a) Pronto (*ready*).
 - b) Em execução (*running*).
 - c) Em espera (*waiting*) e liberta a CPU.
 - d) Em espera (*waiting*), mas não liberta a CPU.
- 11** – No escalonamento de processos baseado em prioridades, a inversão de prioridades refere-se a:
- a) Uma redução da prioridade de um processo por este bloquear constantemente em operações de I/O.
 - b) Um aumento da prioridade de um processo para que este liberte mais rapidamente um recurso requerido por um processo de maior prioridade.
 - c) Uma situação em que um processo de maior prioridade é obrigado a esperar por um processo de menor prioridade.
 - d) Uma atribuição de prioridades aos processos pela ordem inversa dos seus tempos de execução.
- 12** – Relativamente ao escalonamento multinível com realimentação por filas, pode-se afirmar que:
- a) Cada fila deve obrigatoriamente aplicar o mesmo algoritmo de escalonamento, podendo variar apenas os parâmetros, como o tamanho do time quantum entre filas.
 - b) É adequado para situações em que os parâmetros de funcionamento dos processos a serem escalonados são bem conhecidos antecipadamente.
 - c) Adapta-se dinamicamente ao comportamento dos processos durante a execução.
 - d) Exige que cada processo seja atribuído a uma fila fixa e nunca mude de fila.
- 13** – A estratégia de escalonamento que impede que um processo em execução monopolize o CPU até ao fim do seu código ou até o libertar voluntariamente é denominada por:
- a) Escalonamento não preemptivo.
 - b) Escalonamento preemptivo.
 - c) Escalonamento por prioridades fixas.
 - d) Escalonamento por prioridades dinâmicas.
- 14** – Aplicar ao algoritmo de escalonamento *Round Robin*:
- a) Um *time quantum* muito grande converte-o na prática no algoritmo *First In First Out*.
 - b) Um *time quantum* muito grande aumenta consideravelmente a performance do sistema.
 - c) Um *time quantum* muito grande aumenta consideravelmente o custo (*overhead*) do escalonamento dos processos.
 - d) Um *time quantum* muito grande converte-o na prática no algoritmo *Shortest Job First*.

15 – Considere o seguinte excerto de código:

<pre>void *f1(void *arg){ printf("S1\n"); pthread_exit(NULL); }</pre>	<pre>for(i = 0; i < 3; i++){ pid = fork(); if (pid > 0){ execlp("ls", "ls", NULL); pid = fork(); } else{ pthread_create(&t[i], NULL, f1, NULL); printf("S1\n"); } printf("S2\n"); }</pre>
---	---

Quantas vezes irá ser impresso "S1"? Assuma que a invocação da função `execlp()` nunca falha.

- a) 18.
- b) 6.
- c) 36.
- d) 3.

16 – Qual das seguintes afirmações melhor descreve o fluxo de controlo excecional num sistema operativo?

- a) O fluxo de controlo excecional ocorre quando um processo a executar em *user-space* altera diretamente o fluxo de execução para *kernel-space* sem qualquer forma de mediação.
- b) O fluxo de controlo excecional diz respeito a eventos como interrupções, *traps* e exceções que fazem com que o CPU mude de *user-space* para *kernel-space* para lidar com condições ou eventos inesperados.
- c) O fluxo de controlo excecional é um mecanismo que garante que todos os processos recebam igual tempo de CPU, interrompendo preemptivamente os processos em execução em intervalos fixos.
- d) O fluxo de controlo excecional só ocorre durante o encerramento do sistema operativo para garantir que todos os processos são corretamente terminados.

17 – Considere os seguintes processos P1 e P2 que executam num único processador. Assuma que existem dois semáforos (S1, S2), em que S1 é inicializado a um (1) e S2 é inicializado a zero (0), e que as funções `up(s)` e `down(s)` permitem incrementar e decrementar um semáforo, respetivamente, de forma atómica.

P1	P2
...	...
<code>down(S1);</code>	<code>down(S2);</code>
<code>up(S2);</code>	<code>/* Executa bloco B */</code>
<code>/* Executa bloco A */</code>	<code>down(S1);</code>
<code>up(S1);</code>	<code>up(S2);</code>
<code>/* Executa bloco C */</code>	<code>/* Executa bloco D */</code>

Indique qual das seguintes afirmações é verdadeira:

- a) P1 executa sempre o bloco C antes de P2 executar o bloco D.
- b) P1 nunca executa o bloco C antes de P2 executar o bloco D.
- c) Nada pode ser garantido em relação à ordem de execução dos blocos C e D.
- d) P2 executa sempre o bloco B antes de P1 executar o bloco A.

18 – Considere um sistema com um único processador e um algoritmo de escalonamento *Shortest Remaining Time First*. Considerando os seguintes tempos de chegada ao sistema e perfis de execução para os processos P1, P2 e P3

Processo	Perfil do processo	Tempo de chegada
P1	111I1	2
P2	2222I2	1
P3	333I33	0

em que um 1, 2 ou 3 representa, respetivamente, o processo P1, P2 ou P3 em execução durante uma unidade de tempo e I representa o bloqueio do processo em I/O. Indique a sequência de execução destes processos, sabendo que na solução o símbolo “-” significa que o processador não está a executar qualquer processo.

- a) 111-1222-2333-333
- b) 3331331121222-2
- c) 33322211133321
- d) 321112122323-33

19 – Assuma o mesmo conjunto de processos, respetivos perfis de execução e tempos de chegada, mas um algoritmo de **escalonamento preemptivo de prioridades fixas**, em que os processos têm prioridades (P1=1 (mais alta); P2=3 (mais baixa); P3=2). Indique a sequência de escalonamento para estes processos (note-se que o símbolo “-” significa que o processador não está a executar qualquer processo).

- a) 33111-133-332222-2
- b) 32111212323-333
- c) 3311131332222-2
- d) 33322211133321

20 – O código seguinte apresenta uma solução para o problema dos “Produtores/Consumidores”:

Produtor	Consumidor
<pre> 1. void write(int elem){ 2. pthread_mutex_lock(&mux); 3. while(n_elems == MAX) 4. pthread_cond_wait(&nf, &mux); 5. buffer[write_pos++] = elem; 6. ... 7. pthread_mutex_unlock(&mux); 8. }</pre>	<pre> 1. void read(int &elem){ 2. pthread_mutex_lock(&mux); 3. ... 4. n_elem--; 5. if(n_elem == MAX - 1) 6. pthread_cond_broadcast(&nf); 7. printf("SCOMP\n"); 8. pthread_mutex_unlock(&mux); 9. }</pre>

Admita que existem duas *threads* produtoras bloqueadas na linha 4 e uma *thread* consumidora a executar a linha 6.

- a) A linha 7 da *thread* consumidora será sempre executada antes de qualquer uma das *threads* produtoras conseguir executar a sua linha 5.
- b) A linha 5 das *threads* produtoras será sempre executada em simultâneo pelas duas *threads* antes da *thread* consumidora executar a sua linha 7.
- c) Cada uma das *threads* produtoras executa sempre de forma exclusiva a sua linha 5 antes da *thread* consumidora executar a sua linha 7.
- d) Não é possível garantir qualquer ordem de execução entre a linha 7 da *thread* consumidora e da linha 5 das *threads* produtoras.

SISTEMAS DE COMPUTADORES – 2023/2024
Exame Época Normal

Apenas autorizada a consulta da folha oficial

Nome: _____ Nº _____

Parte prática
(6/20 valores)

NOTAS:

- 1. Responder numa folha separada, devidamente identificada.**
- 2. Não é necessário indicar o nome das bibliotecas usadas na resolução.**

Implemente um programa em C que, através do uso de *threads*, implemente um cenário de múltiplos escritores e múltiplos leitores. Haverá N escritores e M leitores (M e N podem ser estáticos ou definidos pelo utilizador).

Num acesso concorrente a uma área de memória partilhada, é fundamental garantir a sincronização adequada entre as operações de leitura e escrita:

- **leitor** – responsável por ler duas *strings* de uma estrutura presente na *heap*:
 - Os leitores não modificam a área de memória partilhada, permitindo que **vários leitores a acedam simultaneamente**.
 - Os **leitores têm prioridade** sobre os escritores.
 - Cada leitor deve imprimir as *strings* lidas, juntamente com a contagem atual de leitores ativos.
- **escritor** – responsável por escrever na área de memória partilhada na *heap*:
 - Escreve o ID da *thread* e a hora atual.
 - Apenas **um escritor** pode aceder à área de memória partilhada em qualquer momento.
 - Os escritores só podem aceder à área de memória partilhada **quando não há leitores ativos**.
 - Cada escritor imprime a contagem de escritores, bem como a contagem de leitores ativos.